LAPORAN PRAKTIKUM PEMROGRAMAN WEB

PERTEMUAN 6

CRUD CREATE DAN READ DENGAN PDO

Disusun Oleh:

ANDRIAN YUZA SWANDA

2401020157

Dosen Pengampu:

Nolan Efranda, M.Kom.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN

UNIVERSITAS MARITIM RAJA ALI HAJI

2026

I. TUJUAN

Pada praktikum keenam ini saya melengkapi aplikasi Laravel dari pertemuan kelima dengan operasi Create dan Read memakai PDO. Create dipakai untuk menambah data mahasiswa baru dari form, sedangkan Read dipakai untuk menampilkan daftar mahasiswa beserta nama program studinya. Sasaran saya adalah melatih validasi masukan di sisi server, mencegah NIM atau email ganda, memakai prepared statement pada perintah SELECT dan INSERT, serta memberi umpan balik yang jelas ketika data berhasil disimpan atau ditolak.

II. KEBUTUHAN PERANGKAT

Perangkat yang saya pakai masih sama seperti pertemuan sebelumnya, hanya saja kali ini saya menambah uji coba pada form masukan dan tabel data.

Perangkat	Keterangan
Laravel	Proyek praktikum_web_2401020157 hasil Modul 5
PHP	8.2.12 dengan ekstensi pdo_mysql
Basis data	MySQL/MariaDB pada XAMPP, basis data praktikum_web_2401020157
Editor	Visual Studio Code
Terminal	PowerShell pada Windows
Browser	Google Chrome
Server lokal	php artisan serve pada http://127.0.0.1:8000
Git	Git dan akun GitHub untuk commit

III. DATA AWAL

Praktik ini saya mulai dari tiga data mahasiswa yang sudah ada sejak pertemuan kelima. Saya memeriksanya lewat terminal MySQL dengan perintah SELECT yang menggabungkan

tabel mahasiswa dan program_studi. Salah satu dari tiga data itu adalah data saya sendiri dengan NIM 2401020157, sehingga syarat menambahkan data dengan identitas masing-masing sudah terpenuhi sejak pertemuan kelima dan datanya tetap tersedia pada praktikum ini.

```
USE praktikum_web_2401020157;
SELECT m.nim, m.nama, m.email, m.usia, p.nama_prodi
FROM mahasiswa AS m
JOIN program_studi AS p ON p.id = m.program_studi_id
ORDER BY m.nim;
```

NIM	Nama	Email	Usia	Program Studi
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

IV. ALUR CREATE DAN READ

Alur kerjanya saya gambarkan seperti berikut. Route GET membaca dua kumpulan data, yaitu daftar mahasiswa dan daftar program studi, lalu mengirim keduanya ke Blade. Form kemudian dikirim lewat POST, divalidasi, diperiksa duplikatnya, baru disimpan dengan prepared statement INSERT. Setelah itu pengguna diarahkan kembali ke halaman yang sama agar umpan balik bisa ditampilkan melalui session.

```
Browser buka /mahasiswa
  -> Route GET ambil PDO, prepare(SELECT mahasiswa JOIN prodi)
  -> prepare(SELECT prodi)
  -> Blade menampilkan form dan tabel
  -> Pengguna isi form lalu kirim POST /mahasiswa
  -> validate() memeriksa field wajib dan format
  -> cek duplikat COUNT(*) pada NIM atau email
  -> INSERT prepared statement
  -> redirect + feedback session
  -> tabel menampilkan data terbaru
```

V. KODE ROUTE GET

Route GET /mahasiswa bertugas untuk operasi Read. Ada dua prepared statement di dalamnya, satu untuk membaca mahasiswa beserta nama program studinya dan satu lagi untuk membaca daftar program studi yang dipakai sebagai pilihan pada form. Seluruh proses saya bungkus dengan try-catch supaya kegagalan koneksi tidak menampilkan detail teknis ke pengguna.

```
Route::get('/mahasiswa', function () {
    try {
        $pdo = DB::connection()->getPdo();
        $pernyataanMahasiswa = $pdo->prepare(
            'SELECT m.nim, m.nama, m.email, m.usia,
                    p.nama_prodi
             FROM mahasiswa AS m
             JOIN program_studi AS p
               ON p.id = m.program_studi_id
             ORDER BY m.nim'
        );
        $pernyataanMahasiswa->execute();
        $daftarMahasiswa = $pernyataanMahasiswa->fetchAll(
            \PDO::FETCH_ASSOC
        );
        $pernyataanProdi = $pdo->prepare(
            'SELECT id, nama_prodi
             FROM program_studi
             ORDER BY nama_prodi'
        );
        $pernyataanProdi->execute();
        $daftarProgramStudi = $pernyataanProdi->fetchAll(
            \PDO::FETCH_ASSOC
        );
        return view('mahasiswa', compact(
            'daftarMahasiswa',
            'daftarProgramStudi'
        ));
    } catch (\Throwable $error) {
        report($error);
        return response(
            'Data tidak dapat dibaca. Periksa koneksi dan query.',
            500
        );
    }
})->name('mahasiswa.index');
```

VI. KODE ROUTE POST

Route POST /mahasiswa menangani Create. Data dari form divalidasi, dicek duplikatnya lewat SELECT COUNT(*), lalu disimpan dengan prepared statement INSERT. Setelah itu pengguna diarahkan kembali ke daftar mahasiswa dengan pesan berhasil.

```
Route::post('/mahasiswa', function (Request $request) {
    $data = $request->validate([
        'nim' => ['required', 'string', 'max:20'],
        'nama' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'max:100'],
        'usia' => ['required', 'integer', 'min:15', 'max:100'],
        'program_studi_id' => ['required', 'integer'],
    ], [
        'nim.required' => 'NIM wajib diisi.',
        'nama.required' => 'Nama wajib diisi.',
        'email.required' => 'Email wajib diisi.',
        'email.email' => 'Format email tidak valid.',
        'usia.required' => 'Usia wajib diisi.',
        'usia.integer' => 'Usia harus berupa angka.',
        'program_studi_id.required' => 'Program studi wajib dipilih.',
    ]);
    try {
        $pdo = DB::connection()->getPdo();
        $cekDuplikat = $pdo->prepare(
            'SELECT COUNT(*) FROM mahasiswa
             WHERE nim = :nim OR email = :email'
        );
        $cekDuplikat->execute([
            'nim' => $data['nim'],
            'email' => $data['email'],
        ]);
        if ((int) $cekDuplikat->fetchColumn() > 0) {
            return back()->withInput()->with(
                'gagal',
                'NIM atau email sudah digunakan.'
            );
        }
        $simpan = $pdo->prepare(
            'INSERT INTO mahasiswa
             (nim, nama, email, usia, program_studi_id)
             VALUES (:nim, :nama, :email, :usia, :program_studi_id)'
        );
        $simpan->execute([
            'nim' => $data['nim'],
            'nama' => $data['nama'],
            'email' => $data['email'],
            'usia' => $data['usia'],
            'program_studi_id' => $data['program_studi_id'],
        ]);
        return redirect()->route('mahasiswa.index')->with(
            'berhasil',
```

```
            'Data mahasiswa berhasil ditambahkan.'
        );
    } catch (\Throwable $error) {
        report($error);
        return back()->withInput()->with(
            'gagal',
            'Data gagal disimpan. Periksa koneksi dan data.'
        );
    }
})->name('mahasiswa.store');
```

VII. KODE BLADE

Tampilan saya buat polos tanpa CSS. Blade di bawah memuat token CSRF, pesan berhasil atau gagal dari session, daftar error validasi, form dengan old() supaya isian tidak hilang, pilihan program studi memakai @selected, dan tabel data memakai @forelse serta @empty.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Data Mahasiswa</title>
</head>
<body>
    <h1>Data Mahasiswa</h1>
    @if (session('berhasil'))
        <p>{{ session('berhasil') }}</p>
    @endif
    @if (session('gagal'))
        <p>{{ session('gagal') }}</p>
    @endif
    @if ($errors->any())
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    @endif
    <h2>Tambah Mahasiswa</h2>
    <form action="{{ route('mahasiswa.store') }}" method="POST">
        @csrf
        <p>NIM<br><input type="text" name="nim" value="{{ old('nim') }}"></p>
        <p>Nama<br><input type="text" name="nama" value="{{ old('nama') }}"></p>
        <p>Email<br><input type="email" name="email" value="{{ old('email')
}}"></p>
        <p>Usia<br><input type="number" name="usia" value="{{ old('usia') }}"></p>
        <p>Program Studi<br>
            <select name="program_studi_id">
```

```
                <option value="">-- Pilih Program Studi --</option>
                @foreach ($daftarProgramStudi as $programStudi)
                    <option value="{{ $programStudi['id'] }}"
                        @selected(old('program_studi_id') == $programStudi['id'])>
                        {{ $programStudi['nama_prodi'] }}
                    </option>
                @endforeach
            </select>
        </p>
        <button type="submit">Tambah Data</button>
    </form>
```

Bagian tabel di bawah ini memakai @forelse untuk mengulang baris data dan @empty ketika belum ada data yang bisa ditampilkan.

```
    <h2>Daftar Mahasiswa</h2>
    <table border="1" cellpadding="6" cellspacing="0">
        <tr>
            <th>NIM</th><th>Nama</th><th>Email</th>
            <th>Usia</th><th>Program Studi</th>
        </tr>
        @forelse ($daftarMahasiswa as $mahasiswa)
            <tr>
                <td>{{ $mahasiswa['nim'] }}</td>
                ...
            </tr>
        @empty
            <tr><td colspan="5">Data belum tersedia.</td></tr>
        @endforelse
    </table>
</body>
</html>
```

VIII. DATA UJI

Untuk pengujian data valid saya memakai satu data baru seperti tabel di bawah. Data ini saya pilih berbeda dari tiga data awal supaya hasilnya mudah dilihat pada tabel.

Field	Nilai
NIM	2201010003
Nama	Rina Kurnia
Email	rina@example.com
Usia	20
Program Studi	Teknik Informatika

IX. TABEL SKENARIO DAN HASIL

Saya menguji enam skenario, yaitu halaman awal, data valid, field wajib kosong, email tidak valid, NIM duplikat, dan kegagalan koneksi. Hasil tiap skenario saya catat pada tabel berikut.

No	Skenario	Hasil
1	Buka /mahasiswa	Form tampil kosong dan tabel berisi tiga data Modul 5.
2	Kirim data valid 2201010003	Pesan "Data mahasiswa berhasil ditambahkan." tampil dan tabel menjadi empat baris.
3	Kirim ulang NIM 2201010003 dengan email berbeda	Pesan "NIM atau email sudah digunakan." tampil; jumlah data tetap empat.
4	Kirim data dengan nama dikosongkan	Pesan "Nama wajib diisi." tampil dan data tidak disimpan.
5	Kirim email tanpa format yang benar	Pesan "Format email tidak valid." tampil dan data tidak disimpan.
6	Koneksi basis data dibuat gagal	Pesan "Data tidak dapat dibaca. Periksa koneksi dan query." tampil; detail dicatat report().

X. BUKTI TANGKAPAN LAYAR

Semua bukti pengujian saya tempel di bawah ini sesuai urutan skenario pada tabel sebelumnya.

Data Mahasiswa

Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

Daftar Mahasiswa

NIM	Nama	Email	Usia	Program Studi
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

Gambar 1. Halaman awal Create dan Read: form kosong dan tiga data Modul 5.

Data Mahasiswa

Data mahasiswa berhasil ditambahkan.

Tambah Mahasiswa

NIM

Nama

Email

Usia

Program Studi
-- Pilih Program Studi --

Tambah Data

Daftar Mahasiswa

NIM	Nama	Email	Usia	Program Studi
2201010003	Rina Kurnia	rina@example.com	20	Teknik Informatika
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

Gambar 2. Data valid Rina Kurnia berhasil ditambahkan sehingga tabel menjadi empat baris.

Data Mahasiswa

NIM atau email sudah digunakan.

Tambah Mahasiswa

NIM
2201010003

Nama
Rina Kurnia

Email
rina.baru@example.com

Usia
20

Program Studi
Teknik Informatika

Tambah Data

Daftar Mahasiswa

NIM	Nama	Email	Usia	Program Studi
2201010003	Rina Kurnia	rina@example.com	20	Teknik Informatika
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

Gambar 3. NIM duplikat ditolak dan nilai form tetap tampil lewat old().

Data Mahasiswa

- Nama wajib diisi.

Tambah Mahasiswa

NIM
2201010004

Nama

Email
dewi@example.com

Usia
20

Program Studi
Teknik Informatika

Tambah Data

Daftar Mahasiswa

NIM	Nama	Email	Usia	Program Studi
2201010003	Rina Kurnia	rina@example.com	20	Teknik Informatika
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

Gambar 4. Nama dikosongkan sehingga validasi menolak data.

Data Mahasiswa

- Format email tidak valid.

Tambah Mahasiswa

NIM
2201010004

Nama
Dewi Lestari

Email
dewi.example.com

Usia
20

Program Studi
Teknik Informatika

Tambah Data

Daftar Mahasiswa

NIM	Nama	Email	Usia	Program Studi
2201010003	Rina Kurnia	rina@example.com	20	Teknik Informatika
2401020045	Muhammad Fauzi	fauzi@example.com	19	Sistem Informasi
2401020151	Christian Aprilio Sihite	christian@example.com	20	Teknik Informatika
2401020157	Andrian Yuza Swanda	andrian.yuza@example.com	21	Teknik Informatika

Gambar 5. Email tidak valid sehingga data tidak disimpan.

Data tidak dapat dibaca. Periksa koneksi dan query.

Gambar 6. Kegagalan koneksi menampilkan pesan aman dari blok catch.

XI. DEBUGGING DAN PERBAIKAN

Waktu pertama kali menguji data duplikat, saya sempat bingung karena pesan penolakan muncul tetapi data lama malah hilang dari form. Setelah saya periksa, ternyata saya lupa

menambahkan withInput() pada cabang kegagalan. Setelah ditambahkan, isian form tetap tampil dan bisa langsung saya perbaiki.

Kendala kedua muncul saat menguji kegagalan koneksi. Saya mengubah nama basis data pada file .env, tetapi yang tampil bukan pesan dari blok catch, melainkan halaman debug Laravel. Penyebabnya adalah session disimpan di basis data, sehingga middleware session gagal lebih dahulu sebelum route dieksekusi. Untuk keperluan pengujian saya ganti SESSION_DRIVER menjadi file, jalankan ulang php artisan config:clear, baru pesan aman "Data tidak dapat dibaca. Periksa koneksi dan query." muncul dengan status 500. Setelah selesai saya kembalikan konfigurasinya.

Satu hal kecil lagi, input email dengan tipe email menahan pengiriman form di browser ketika nilainya tidak valid. Supaya validasi server-side ikut teruji, saya matikan validasi bawaan browser pada saat pengujian tersebut.

XII. REVIEW SEJAWAT DAN PERBAIKAN

Alur Create dan Read, pemeriksaan duplikat, serta hasil pengujian saya minta diperiksa oleh rekan saya, Muhammad Fauzi. Setelah mencoba menambah data dan mengirim NIM yang sama, rekan saya memberi satu saran, yaitu menambahkan batas panjang maksimum pada validasi NIM, nama, dan email agar data yang masuk tidak berlebihan. Saran tersebut saya tindak lanjuti dengan menambahkan aturan max:20 pada NIM serta max:100 pada nama dan email, lalu saya uji ulang dengan data valid, NIM duplikat, dan email tidak valid. Hasilnya tetap sesuai: data valid tersimpan, sedangkan data duplikat dan tidak valid ditolak.

XIII. KENDALA DAN SOLUSI

Kendala pertama adalah isian form yang hilang ketika data ditolak. Solusinya saya pakai back()->withInput() dan helper old() pada setiap kolom. Kendala kedua adalah halaman debug yang menutupi pesan error buatan saya sendiri ketika koneksi dibuat gagal. Solusinya saya pindahkan session ke driver file khusus saat pengujian, lalu saya kembalikan ke driver semula setelah pengujian selesai.

XIV. KESIMPULAN

Praktikum keenam ini berhasil menambahkan operasi Create dan Read pada aplikasi Laravel saya. Data baru bisa disimpan lewat form yang dilindungi token CSRF, diperiksa validasinya, dicegah dari duplikasi, dan langsung tampil pada tabel tanpa CSS. Prepared statement membuat nilai masukan tidak pernah menyatu ke dalam teks SQL, sedangkan rangkaian validasi, pengecekan duplikat, dan try-catch menjaga integritas data serta memberi pesan yang aman bagi pengguna. Operasi Update dan Delete akan saya lanjutkan pada pertemuan berikutnya.

XV. TAUTAN REPOSITORY DAN LAMPIRAN SOURCE CODE

1. Repository GitHub: https://github.com/Zaa0/praktikum-web-2401020157
2. Commit: Pertemuan 6 CRUD Create Read
3. Source code yang dikumpulkan:
    - routes/web.php: route GET /mahasiswa dan POST /mahasiswa dengan PDO prepared statement, validasi, dan pengecekan duplikat.
    - resources/views/mahasiswa.blade.php: form Create dengan CSRF, old(), pesan feedback, dan tabel data tanpa CSS.
    - .env.example: dokumentasi variabel koneksi tanpa kata sandi (file .env tidak di-commit).